

Test Cases

A friendly, beginner-friendly guide
(with examples from our Jest project)

Project: `test-cases-lab` • Stack: Node.js + Jest
Prepared: 22 May 2026

1. What is a Test Case?

A **test case** is a small experiment we run on our code to check that it behaves the way we expect. Think of it like a science lab:

- **Input** — what we feed into the function.
- **Action** — calling the function.
- **Expected output** — what we think should come back.

If the actual output matches what we expected, the test **passes**. If not, it **fails** — and Jest will tell us exactly where and why.

Why bother? Tests give us confidence to change code without breaking things. They are a safety net we build for our future selves.

The basic shape of a Jest test

```
test("description of what we are checking", () => {  
  const result = add(2, 3);  
  expect(result).toBe(5);  
});
```

Every test follows the same friendly pattern called **AAA**:

1. **Arrange** — set up inputs.
2. **Act** — call the function.
3. **Assert** — check the result with `expect(...)`.

2. Types of Tests We Use

In our project we have three categories of tests. They each answer a different question:

Type	Question it answers	Example in our repo
Unit	Does this one function work on its own?	<code>tests/unit.test.js</code>
Negative	Does the code fail <i>correctly</i> on bad input?	<code>tests/negative.test.js</code>
Integration	Do multiple pieces work together?	<code>tests/integration.test.js</code>

3. Unit Tests — testing one small thing

A **unit test** tests the smallest unit of code — usually a single function — in isolation. No database, no network, no other modules.

From our codebase: `src/calculator.js`

```
function add(a, b) {  
  return a + b;  
}  
  
function multiply(a, b) {  
  return a * b;  
}
```

The unit test: `tests/unit.test.js`

```
const { add, subtract, multiply } = require("../src/calculator");  
  
describe("Unit Tests - Calculator", () => {  
  test("add(2, 3) should return 5", () => {  
    expect(add(2, 3)).toBe(5);  
  });  
  
  test("multiply(3, 4) should return 12", () => {  
    expect(multiply(3, 4)).toBe(12);  
  });  
});
```

What's happening: We call `add(2, 3)` and assert (claim) that the result is exactly `5`. If `add` is broken, this test fails immediately — before anything else uses it.

Heads-up: In our current file, one test claims `add(2, 3)` should be `9`. That's an intentionally failing test to demonstrate what a red test looks like. The correct value is `5`.

When to write a unit test

- The function has clear inputs and outputs (pure logic).
- You want fast feedback — unit tests run in milliseconds.
- You want to lock in behavior before refactoring.

4. Negative Tests — making sure bad input is rejected

A **negative test** checks that the code *fails the right way* when given invalid input. We are not trying to make the test pass by being nice to the function — we are deliberately feeding it junk.

From our codebase: `src/userService.js`

```
function registerUser(name, email) {
  if (!name || !email) {
    throw new Error("Name and email are required");
  }
  if (!email.includes("@")) {
    throw new Error("Invalid email format");
  }
  // ... otherwise create the user
}
```

The negative test: `tests/negative.test.js`

```
test("divide(10, 0) should throw 'Cannot divide by zero'", () => {
  expect(() => divide(10, 0)).toThrow("Cannot divide by zero");
});

test("registerUser with invalid email should throw", () => {
  expect(() => registerUser("Ram", "ramgmail.com"))
    .toThrow("Invalid email format");
});
```

Notice that we wrap the call in an arrow function: `() => divide(10, 0)`. This is because `toThrow` needs a function it can call itself — so Jest can catch the error and inspect it.

What counts as a negative case?

- Empty strings, `null`, `undefined`.
- Numbers out of range (zero divisor, negative ages).
- Wrong types (string where a number was expected).
- Malformed values (email without `@`).

5. Integration Tests — testing things together

A **unit test** looks at one function. An **integration test** checks that several functions, modules, or layers cooperate correctly.

In our project, `registerUser`, `getUsers`, and `findUserByEmail` all share the same in-memory `users` array. A unit test cannot tell us if that array is wired up correctly across calls — but an integration test can.

From our codebase: `tests/integration.test.js`

```
describe("Integration Tests - User Registration Flow", () => {
  beforeEach(() => {
    clearUsers(); // reset state so tests do not leak into each other
  });

  test("should register multiple users and find them by email", () => {
    registerUser("Ram", "ram@gmail.com");
    registerUser("Sita", "sita@gmail.com");

    expect(getUsers().length).toBe(2);

    const found = findUserByEmail("sita@gmail.com");
    expect(found.name).toBe("Sita");
  });
});
```

What this proves:

- `registerUser` actually saves into the same store that `getUsers` reads from.
- IDs increment correctly across calls.
- Lookup by email finds the right record.

A single function passing in isolation cannot guarantee any of that.

Why `beforeEach(clearUsers)` ?

Tests should be **independent**. If one test leaves three users in the array, the next test will see them too — and may fail or pass for the wrong reason. `beforeEach` runs before every single test in the block and resets the world to a clean state.

6. A Sneak Peek: E2E Tests

We do not have these in our repo yet, but they are good to know about. **End-to-End (E2E) tests** drive the entire application the way a real user would.

Test type	What runs	Speed
Unit	One function in memory	Milliseconds
Integration	Several functions / modules together	Fast — usually under a second
E2E	Real browser → real server → real database	Seconds to minutes

Example E2E scenario (conceptual)

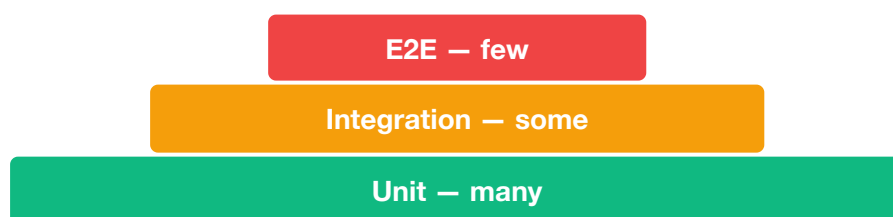
1. Open the browser at `http://localhost:3000/signup`.
2. Type a name and email into the form.
3. Click the *Register* button.
4. Assert that the page shows "Welcome, Ram".

Popular E2E tools include:

- **Cypress** — friendly UI, great for frontend devs.
- **Playwright** — fast, multi-browser, made by Microsoft.
- **Selenium** — the original; supports many languages.

Trade-off: E2E tests give the highest confidence ("the whole app works!") but they are slow and can be flaky. We use them sparingly — only for the most important user journeys.

The Test Pyramid



A healthy project has lots of unit tests at the base, fewer integration tests in the middle, and only a handful of E2E tests at the top.

7. Running the Tests

From the project root:

```
npm install      # first time only
npm test         # runs every test in /tests
```

Jest prints a green tick for passing tests and a red cross for failing ones. A typical output looks like:

```
PASS  tests/integration.test.js
PASS  tests/negative.test.js
FAIL  tests/unit.test.js
  ● Unit Tests – Calculator › add(2, 3) should return 5

    expect(received).toBe(expected)

    Expected: 9
    Received: 5
```

The error tells us *which* test failed, *what* we expected, and *what* we actually got — usually enough to fix the bug in seconds.

8. Good Habits Cheat-Sheet

Do

- Give tests descriptive names — they double as documentation.
- Test one thing per test. If it fails, you know exactly what broke.
- Use `beforeEach` to keep tests isolated.
- Cover the happy path *and* the sad path (negative cases).

Avoid

- Tests that depend on each other's leftover state.
- Tests with no assertion — they pass silently and prove nothing.
- Hard-coded sleeps / waits — prefer event-based waits.
- Testing the framework instead of your own logic.

9. Glossary

Term	Meaning
<code>describe</code>	A group of related tests.
<code>test</code> / <code>it</code>	A single test case.
<code>expect</code>	Start of an assertion.
<code>toBe</code>	Strict equality (<code>===</code>).
<code>toThrow</code>	The given function must throw an error.
<code>beforeEach</code>	Runs before every test in the block.
Assertion	A claim about a value — fails the test if untrue.
Mock	A fake replacement for a real dependency.
Coverage	How much of the source code your tests touch.